

This Lab's Objectives

1. Create a self-signed certificate.
 2. Create a virtual host web site to use the certificate.
 3. Analyse the TCP conversation between client and secure server.
-

Expected Outcomes

After the laboratory session, students should be able to:

- Generate a self-signed SSL certificate.
 - Create an Apache virtual host web site to use the certificate.
 - Analyse secure web traffic with Wireshark.
-

Workplan

I recommend that you buy a hardback A4 laboratory notebook. Everything you do in the lab should be noted down for later consideration and revision. You should use your lab diary to note down items such as commands used, the outcome of each command, and answers to questions posed in these instructions. Alternatively, use CherryTree Notes, or a similar application.

NB this is a very complex lab – please read each instruction carefully – if you miss out any steps, then don't expect to achieve the correct results! You will need to have completed and thoroughly understood all the steps before the next lab session. Remember that there is a lab work wiki available to you on BB. Please use this to share and communicate with you colleagues.

Make sure you have completed all the steps in this worksheet before the next lab session.

Starting assumptions

1. You have completed all the work in previous lab sheets.
 - You have used Netkit *lstart* to launch a coordinated group of several virtual machines.
Specifically, you are aware of how this group is coordinated via the *lab.conf* and *lab.deb* files.
 - You know how to shut down virtual machines using *lcrash* and *lclean*.
 - You have used *ip* and *ping* within the virtual machines.
 - You have saved captured network traffic from a virtual machine using *tcpdump*.
 - You have viewed a packet capture file in the real Linux host using *wireshark* and can drill-down into the display to analyse the protocols in detail.

Activities

Reminder

- You need to make accurate notes of what you do.
- You will complete unfinished activities before the next timetabled lab session.
- You will resolve what you do not understand by conducting your own careful (and ethically sound) experimentation and/or further reading.

Getting Started

2. Create a directory for this lab's work as per previous labs.
3. Download and extract the compressed file ***netkit_lab_ssl-certificates.tar.xz*** from Blackboard.
 - This contains the configuration information for just 2 virtual machines – *ssl-server* and *ssl-client*.
 - *ssl-server* is pre-configured to be a web server, and the apache web service is ready-started for you.
4. Start the lab and check that the two vms can inter-communicate (how?).
5. Use *tcpdump* to capture an HTTP/TCP conversation in the ssl server VM for later analysis and comparison. Use the text-only browser application called “lynx” in the *ssl-client* VM to generate the network traffic. Close the capture when complete.

```
tcpdump -i eth0 -v -s0 -w /hosthome/ctec1704/captures/http2.pcap
```

Self-signed SSL certificates

*NB – the most common mistake made in the following section is to forget that you are creating a certificate in the **ssl-server** virtual machine – not in the host system!*

You are going to create a self-signed certificate as discussed in the SSL lecture. *OpenSSL*, the software that is used to create the certificate, is installed in the Netkit vm by default.

Carefully follow the instructions in this tutorial. If you miss a step, or miss-type a command, your certificate will not work, and you will have to start again.

6. NB these notes are based upon http://www.akadia.com/services/ssh_test_certificate.html, so please spend some time reading the web-page.

It is traditional to store the SSL certificate files in a new directory.

7. In the *ssl-server* web server VM:

```
root@ssl-server:~# mkdir /etc/apache2/ssl
root@ssl-server:~# cd /etc/apache2/ssl
```

Step 1: Generate a Private Key

8. Enter the following.
Make sure you fully understand what each of the parameter values is for.

```
root@ssl-server:/etc/apache2/ssl# openssl genrsa -des3 -out
server.key 2048
```

Output:

```
Generating RSA private key, 2048 bit long modulus (2 primes)
.....++++++
.....++++++
e is 65537 (0x10001)
Enter pass phrase for server.key:
Verifying - Enter pass phrase for server.key:
```

Step 2: Generate a Certificate Signing Request (CSR)

During the generation of the CSR, you will be prompted for several pieces of information. These are the X.509 attributes of the certificate. One of the prompts will be for "Common Name (e.g., YOUR name)". It is important that this field be filled in with the fully qualified domain name of the server to be protected by SSL, eg, if the website to be protected will be *https://evil.org*, then you would enter *public.evil.org* at this prompt.

Enter the following command. As before, ensure that you fully understand each of the parameter values.

```
root@ssl-server:/etc/apache2/ssl# openssl req -new -key
server.key -out server.csr
```

Output:

```
Enter pass phrase for server.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
```

```
Country Name (2 letter code) [AU]:GB
State or Province Name (full name) [Some-State]:Leics
Locality Name (eg, city) []:Leicester
Organization Name (eg, company) [Internet Widgits Pty Ltd]:DMU
Organizational Unit Name (eg, section) []:CompNets
Common Name (eg, YOUR name) []:192.168.150.60
Email Address []:
```

```
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:password
An optional company name []:
```

Step 3: Remove Passphrase from Key

If the server key has a pass-phrase, you will need to enter this each time the server starts, which is not practical in many situations (web servers tend to be remotely hosted). Therefore it is often advantageous to remove the pass phrase. This means that it is critical that this file is only readable by the root user in future!

9. Enter the following.

```
root@ssl-server:/etc/apache2/ssl# cp server.key server.key.org
root@ssl-server:/etc/apache2/ssl# openssl rsa -in server.key.org
-out server.key
```

Outcome:

Enter pass phrase for server.key.org:
writing RSA key

Step 4: Generating a Self-Signed Certificate

If you are creating a commercial web site, you would probably now want your certificate to be signed by a Certificate Authority. However, it is possible to self-sign your certificate, but this will generate an error message in a browser.

To self-sign a certificate that lasts for 120 days only:

```
root@ssl-server:/etc/apache2/ssl#  
openssl x509 -req -days 120 -in server.csr -signkey server.key  
-out server.crt
```

Outcome:

```
Signature ok  
subject=/C=GB/ST=Leics/L=Leicester/O=DMU/OU=MTWA/  
CN=192.168.150.60  
Getting Private key
```

10. Check that you now have the following files in your *ssl* directory:

```
root@ssl-server:/etc/apache2/ssl# ls -l
```

Outcome:

```
total 4  
-rw-r--r-- 1 root root 1229 Feb  8 16:26 server.crt  
-rw-r--r-- 1 root root 1041 Feb  8 16:23 server.csr  
-rw----- 1 root root 1675 Feb  8 16:26 server.key  
-rw----- 1 root root 1743 Feb  8 16:25 server.key.org
```

The next stage is to configure a new, secure, web site, on the *ssl-server* VM.

Lab worksheet

Self-signed Certificates & SSL – Create an SSL-enabled Web Site

Create a new web site

11. Make a new directory `/var/www/ssl/`, then move into the directory. This will be the home directory for your new SSL web site.
12. Copy `/var/www/html/index.html` into this new directory.
13. Edit `/var/www/ssl/index.html` and change the output text to something relevant to SSL.
 - Use the *nano* application to edit files in a Netkit VM.

Configure an Apache virtual host

The new SSL web site will be available as an Apache *virtual host*. This is a technique that allows a single Apache web server to host multiple web-sites.

14. Change directory to `/etc/apache2/sites-available/`. List the files there. Compare this to the list in `/etc/apache2/sites-enabled/`
15. Copy `default-ssl.conf` to `default-ssl.conf.old` – this is in case you make a mistake when editing the file.
16. Open `default-ssl.conf` for editing.
 - NB note the port number in line 2 is 443
17. Change the path `/var/www/html` to be `/var/www/ssl/`
18. Find the two lines in the file
`SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem`
`SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key`
19. Change the values for these two settings to match the path and file names of your two new certificate files (*crt* & *key*).
20. Save and close the file when complete.
21. Move back into the root directory

```
root@ssl-server:/etc/apache2/sites-available# cd ~
```

You now have to *enable* your new web site. This is achieved with the *a2ensite* command.

22. Enter:

```
root@ssl-server:~# a2ensite default-ssl.conf
```

Lab worksheet

Self-signed Certificates & SSL – Create an SSL-enabled Web Site

Output:

Enabling site default-ssl.

To activate the new configuration, you need to run:
`systemctl reload apache2`

23. Check the files in `/etc/apache2/sites-enabled/` again. You should see a new file, which is a *symbolic link* (look this up) pointing to your new web-site.
24. You now need to enable the Apache SSL module:

```
root@ssl-server:~# a2enmod ssl
```

Output:

...

To activate the new configuration, you need to run:
`systemctl restart apache2`

25. You will need to restart your Apache server to force it to read the changes in the updated configuration file.

```
root@ssl-server:~# systemctl restart apache2.service
```

If you have made a mistake along the way, you will see an error message such as:

```
Job for apache2.service failed because the control process exited
with error code.
See "systemctl status apache2.service" and "journalctl -xe" for
details.
```

26. If it doesn't work, then check every step and recreate the certificate, if necessary.
27. When all is working, the certificate can be inspected with the *openssl* command.

```
root@ssl-server:~# openssl x509 -in
/etc/apache2/ssl/server.crt -text
```

Can you see the details you entered?
What other information is displayed?

28. Test your newly created and signed certificate with *lynx* in your *ssl-client VM*. Remember that SSL listens on port 443 (protocol type *https://*). You should see the output from the new version of the *index.html* file.

Lab worksheet

Self-signed Certificates & SSL – Create an SSL-enabled Web Site

Capturing HTTP SSL request for Analysis

29. Use *tcpdump* to capture the packets transmitted from your new secure web-site to your browser and save them as *https1.pcap* file for analysis with Wireshark.
30. Open the new *pcap* file with Wireshark.
 - What packet types can you see in the capture?
 - Compare this HTTPS TCP conversation with the HTTP version.
 - Can you see any differences in the packet payloads?
 - How about differences in the status flags?
31. Repeat the capture again (different output file name), but run *nmap* in the client vm window to scan the web server. Again, analyse the output in wireshark, particularly any status bit settings.
32. Enter the following command (what do the switches do?)

```
root@ssl-client:~# nmap -sV -p 1-65535 192.168.150.60/28
```

(<https://hackertarget.com/nmap-tutorial>)

33. Now review what you have learned in this lab session.

Challenge:

34. All the configuration changes you have made are stored in the *.disk* file. When you delete this file (by crashing the lab), you will lose your settings/progress.

Therefore, save your work by copying the edited files into the relevant directories in the *ssl-server* directory in the host system (hint – the */hostlab* directory will help here). Then, when you restart the lab, all your changes should be copied into the *ssl-server* VM.

Test your new configuration to ensure that the certificate and other configuration files have been correctly installed.

Make sure you have completed all the steps in this worksheet before the next lab session.